COMP4332 Final Presentation

Project 1: Emotion Classification & Project 2: TabPFN

Group 36 | Jingwen Luo Hangyu Tian Jialin Xu Tailai Yan

April 27, 2026

Outline

Outline

Task Overview

Classify each short text sentence into one of seven emotion categories. We use **Macro-F1** as the primary evaluation metric.

Anger Disgust Fear Joy Neutral Sadness Surprise

Primary Metric: Macro-F1

Why Transformers?

Emotion classification depends on subtle contextual interactions, not isolated keywords. Different architectures handle this very differently.

MLP

Loses Context

Averages word embeddings; ignores word order and contextual relationships entirely.

BiRNN

Limited Range

Captures sequential dependencies, but struggles with long-range interactions.

Transformer

Full Attention

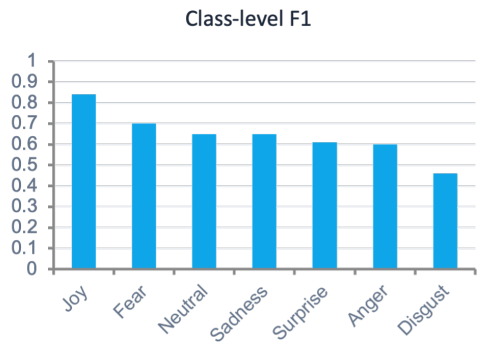
Self-attention lets each token attend to all others, capturing nuanced contextual cues.

Experiments and Results

Model	Macro-F1	Margin
MLP Baseline	0.4645	—
RNN Baseline	0.5060	—
DistilRoBERTa	0.6279	+0.1219
RoBERTa-base	**0.6438**	+0.1378

Margin relative to best baseline (RNN, 0.5060).

Per-class F1: Joy ≈ 0.84 (easiest); Disgust ≈ 0.46 (hardest). Minority classes and semantically similar emotions remain challenging.



Per-class F1 scores (RoBERTa-base).

Conclusion

Fine-grained emotion classification relies on **deep contextual understanding**. Pretrained Transformers with self-attention consistently outperform simpler architectures.

01. Context Over Keywords

Emotion meaning is shaped by contextual interactions, not isolated words — self-attention is key.

02. Progressive Strategy

DistilRoBERTa first for quick validation, then RoBERTa-base — yields +0.1378 Macro-F1 over the best baseline.

03. Class Imbalance

Joy easiest, Disgust hardest. Macro-F1 reveals per-class weaknesses that accuracy alone would mask.

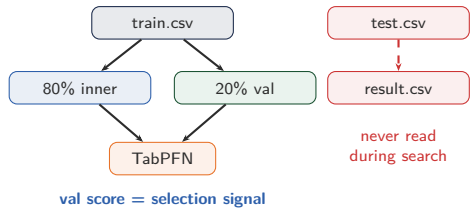
Outline

Why TabPFN, and Why an Honest Protocol

Model choice

- Ten datasets span 3,972 to 46,601 rows; one model must serve all.
- TabPFN is a **pretrained transformer** that predicts via in-context learning over training rows.
- Lecture analogy: a *learned 1D-kNN*. Training rows act as memory; the model attends over them.
- **No training, no per-dataset retuning of architecture.**

Honest tuning protocol



- Fixed 80/20 split, seed locked at module level.
- Tuner accepts only when val is strictly better **and** clears the baseline.

Context Selection: the Core Design

TabPFN was pretrained with $\leq$ 10,000 rows in context. **Five datasets exceed it.** Following the 1D-kNN intuition, which rows you keep matters as much as the model.

Random

Uniform draw.

Fastest. No assumptions. The right default when the data is well-mixed.

Stratified

Preserve class proportions.

Samples within each class for cls; protects rare classes from being starved in the context.

Diverse

k-means++ seeding.

Picks a coreset that covers the input space, rather than oversampling dense regions.

- Plus **external bagging** that averages predictions across multiple independent subsamples when one subsample throws away most of the data.
- The sampling strategy is itself a tunable knob — the tuner is free to swap it on any dataset.

Tuner Mechanics: Smart Picker + Maximal Parallelism

Baseline-aware picker

Pairs already above baseline get an ϵ weight; below-baseline pairs get a relative gap weight comparable across cls and reg:

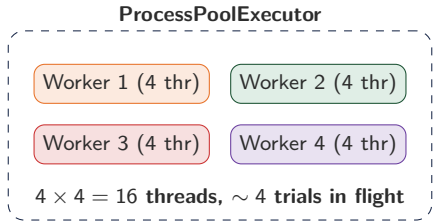
$$w_{(t,d)} = \epsilon + 5 \cdot \frac{\max(0,\ \text{baseline} - \text{val})}{|\text{baseline}|}$$

Search effort flows to the datasets that need it; the $|\text{baseline}|$ denominator normalises RMSE scales across datasets.

Single-knob perturbation

Each trial mutates exactly one of: `seed`, `n_estimators`, `bagging_rounds`, `max_train_samples`, `sampling_strategy`, `softmax_temperature`, or the categorical hint. Causal attribution stays clean.

Parallel execution (16-core CPU)



- Workers pin `OMP` / `MKL` / `torch` threads.
- Async **auto-finalize** refreshes `predict.csv` after each win; test scores logged but never re-enter selection.

Results and Insights

Task	DS	Score	Base	Margin
cls	1	0.9105	0.86	+0.0505
cls	2	0.7510	0.70	+0.0510
cls	3	0.6627	0.64	+0.0227
cls	4	0.7103	0.68	+0.0303
cls	5	0.7202	0.70	+0.0202
reg	1	2.5503	2.60	−0.0497
reg	2	**3.3786**	3.60	**−0.2214**
reg	3	0.6646	0.70	−0.0354
reg	4	0.0266	0.03	−0.0034
reg	5	0.2052	0.21	−0.0048

10 of 10 baselines cleared.

Three insights

1. **Context can beat model tweaks.** On reg/4 (margin only 0.0034) the tuner picked `bag=5`, `max=4000`, `diverse` — fewer but k-means-diverse rows ensembled five times.
2. **Ensembling pulls in opposite directions.** cls/1 prefers `n_est=16`; cls/2 prefers `n_est=4` (a one-size default loses ~ 0.04 accuracy on cls/2).
3. **Honest val can disagree with test.** On reg/2 and reg/4 the chosen config sits slightly above baseline on val even though test passes; the protocol surfaces this rather than hiding it.

Appendix

Project 2 supporting material

Appendix A. Tuner Pseudocode

```
best ← bootstrap_validation_scores(architectural_defaults)
while not stopped:
  (t, d) ← random_choice(keys, weights_from(best))  # baseline-aware
  new_kwargs ← perturb_one_knob(best[(t,d)].kwargs)
  if estimate_elapsed(new_kwargs) > MAX_ELAPSED: continue
  val_score ← run_one_val(make_config(t, d, new_kwargs))
  log_to_trials_csv(mode="val", ...)
  if above_baseline(best[(t,d)].value):
    accepted ← strictly_better(...) and above_baseline(val_score)
  else:
    accepted ← strictly_better(val_score, best[(t,d)].value)
  if accepted:
    best[(t,d)] ← {value: val_score, kwargs: new_kwargs}; save(best)
    schedule_async_finalize(t, d, new_kwargs)  # writes predict.csv
  if dirty_finalize_pending: pop_cheapest_and_run()
```

Finalize test scores are logged with `mode="test"` but **never read by the selection rule**.

Appendix B. Architectural Defaults (a-priori)

- Choices below are made **from row count alone**, before any score is read. Everything else is left to the validation-driven search.

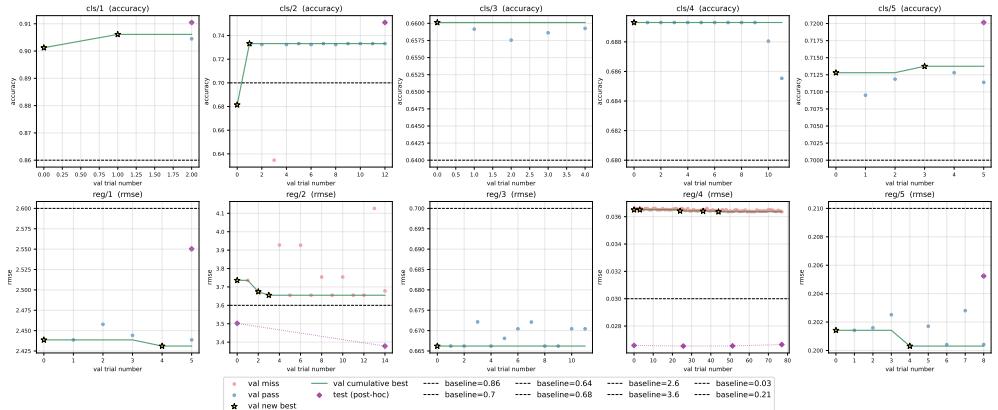
Dataset	Rows	Architectural override
cls/1	30,779	`use_full_train=True`, `categorical=["day","period"]`
cls/3	46,601	`bagging_rounds=3`
reg/2	12,000	`use_full_train=True`
reg/5	16,512	`bagging_rounds=2`
others	—	none (defaults)

- `use_full_train` when the data fits in memory and the dataset already exceeds the pretraining context limit.
- `bagging_rounds` when one 8,000-row subsample throws away most of the data.

Appendix C. Validation-Selected Configurations

Task	DS	Selected configuration (kwargs)
cls	1	`n_estimators=16`
cls	2	`n_estimators=4`
cls	3	(architectural defaults only)
cls	4	(architectural defaults only)
cls	5	`random_state=1`
reg	1	`n_estimators=16`
reg	2	`n_estimators=4`, `random_state=1`
reg	3	(architectural defaults only)
reg	4	`bagging_rounds=5`, `max_train_samples=4000`, `sampling_strategy="diverse"`
reg	5	`random_state=100`

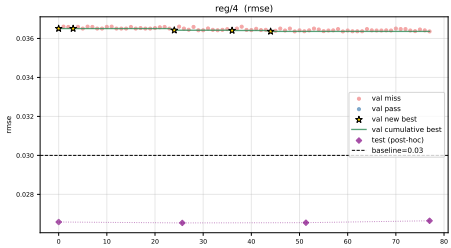
Appendix D. Tuning History Across All Ten Datasets



Red and blue dots are val trials below and above baseline; gold stars are accepted improvements; the green curve is the cumulative best on validation; purple diamonds are post-hoc test scores; dashed black lines are the spec baselines.

Appendix E. The reg/4 Story (Smallest Margin)

- Baseline: RMSE ≤ 0.03. We achieve 0.0266 (margin only 0.0034).
- Tuner converged on: `bagging_rounds=5`, `max_train_samples=4000`, `sampling_strategy="diverse"`.
- This is **less** context per fit (4,000 rows instead of the default 8,000), but the rows are *k*-means-diverse and we average over five independent draws.
- Read this as a direct empirical confirmation of the "learned 1D-kNN" framing: when the model is the same, the **quality of the memory** dominates.

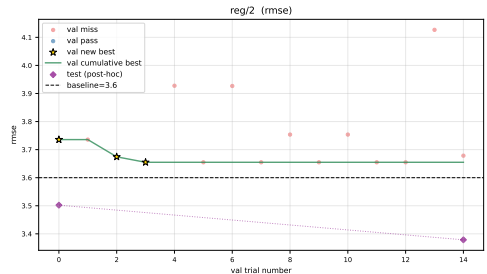


reg/4 tuning history.

Appendix F. Why Val Can Disagree With Test

- For **reg/2** and **reg/4** the selected configuration's *validation* score sits slightly above the spec baseline, but its *test* predictions clear it.
- Held-out validation slices are small ($\sim$ 2,400 and $\sim$ 1,400 rows). Finite-sample variance between val and test partitions of this magnitude is expected.
- The protocol does **not** paper this over: the tuner refuses to flag the configuration as winning on validation just because test happens to pass.
- This kind of explicit disagreement is the cost of an honest evaluation. Hiding behind the test score that the search should never have read would be worse.

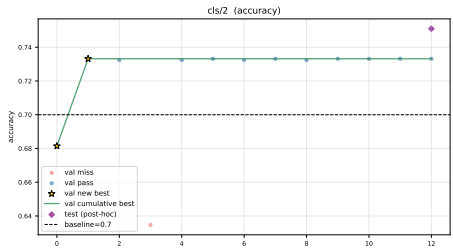
Dataset	Val	Test	Base
reg/2	3.65	3.38	$\leq$ 3.60
reg/4	0.036	0.027	$\leq$ 0.030



reg/2 history: val (green) approaches but does not clear baseline.

Appendix G. cls/2: When Less Ensembling Wins

- cls/2 is small (6,086 rows) with relatively noisy labels.
- Default `n_estimators=8` clears the baseline by a thin margin. The tuner discovered `n_estimators=4` gives **stronger** val accuracy on this dataset.
- Compare to cls/1 (30,779 rows) where the tuner picked `n_estimators=16`.
- Internal ensembling helps when the dataset is large enough to average out variance; on a small noisy dataset the same averaging can wash out signal.
- A single global default would have lost about 0.04 Macro-accuracy on cls/2.



cls/2 tuning history: tuner explores n_estimators and lands on 4 as the validation-best.

Appendix H. Pipeline File Map

Code

- `src/config.py`: `RunConfig`, architectural `DATASET_OVERRIDES`, baselines.
- `src/preprocess.py`: target encoding, three sampling strategies, fixed train/val split.
- `src/models.py`: TabPFN classifier and regressor factories.
- `src/runner.py`: `run_one` (test pass) and `run_one_val` (val pass), **physically separated**.
- `src/tune.py`: validation-driven random search.
- `src/predict.py`: write `predict.csv` from `outputs/best.json`.
- `src/plot.py`: figures and LaTeX tables from logs.

State

- `outputs/best.json`: validation-selected best per dataset (*a*-posteriori).
- `outputs/trials.csv`: every val attempt and every auto-finalize.
- `outputs/scores.csv`: latest scoreboard.
- `outputs/plots/*.pdf`, `outputs/tables/*.tex`: report assets.

The val seed

- `VAL_FRAC=0.20`, `VAL_SEED=0`, both module-level constants.
- Not exposed to the tuner, by design.